```
> ##########################################################################
  ######
  #  photon_cylindrical_TauTimp.mpl
  #  ------------------------------------------------------------------------
  ------
  #  Photon — Cylindrical TauTimp wave equation with axial symmetry.
  #
  #  Solves:
  #    (R/r) * laplacian_cyl(f(r,z,t)) = (1/c^2) f_tt   ,  f(R,0,t) = 0
  #  by separation of variables f = H3(r) H4(z) u(t), reduces the radial
  piece
  #  to the Airy equation, and quantises omega_n via the boundary condition
  #  Ai(-y(R)) = 0.
  #
  #  Then constructs P(r,z,t) satisfying the Maxwell-type system
  #    P_tt - c^2 P_rr = 0  ,  P(R,0,0) = 0
  #  with cylindrical symmetry, a Gaussian envelope, and zeros of P at fixed
  t
  #  on a periodic lattice  r = m * r_b.
  #
  #  Anchors:  R = 3.9228e22 m ,   c = 2.99792458e8 m/s.
  ##########################################################################
  ######

  restart:
  with(plots):
  with(plottools):
  interface(displayprecision = 10):

  printf("============================================================\n");
  printf("SECTION 0 -- Physical constants and TauTimp anchors\n");
  printf("============================================================\n");

  R_val := 3.9228e22:        # m   cosmological TauTimp radius
  c_val := 2.99792458e8:     # m/s
  T_R   := R_val / c_val:    # characteristic time = R/c

  printf("R        = %.6e m\n", R_val);
  printf("c        = %.6e m/s\n", c_val);
  printf("R/c      = %.6e s\n", T_R);
  printf("R^2/c^2  = %.6e s^2\n\n", R_val^2/c_val^2);


  printf("============================================================\n");
  printf("SECTION 1 -- Cylindrical Laplacian (axisymmetric, no theta)\n");
  printf("============================================================\n");

  # Axisymmetric cylindrical Laplacian: Δf = f_rr + (1/r) f_r + f_zz
```

```
LapCyl := f -> diff(f,r,r) + (1/r)*diff(f,r) + diff(f,z,z):

# Master PDE
master_PDE := (R_sym/r) * LapCyl(F(r,z,t)) = (1/c_sym^2) * diff(F(r,z,t),
t, t);
print(master_PDE);

printf("\n");


printf("============================================================\n");
printf("SECTION 2 -- Separation f = H3(r) H4(z) u(t)\n");
printf("============================================================\n");

# Substitute ansatz
ansatz := F(r,z,t) = H3(r) * H4(z) * u(t):
PDE_sub := eval((R_sym/r)*LapCyl(H3(r)*H4(z)*u(t)) - (1/c_sym^2)*diff(H3
(r)*H4(z)*u(t), t, t)):
PDE_sub := simplify(PDE_sub);

# Divide by H3 H4 u: write the separated form by hand
printf("After dividing by H3*H4*u, the separated form is:\n");
printf("\n");
printf("  (R/r)[ H3''/H3 + (1/r) H3'/H3 + H4''/H4 ] = (1/c^2) u''/u\n\n");

printf("Time piece:    u''(t)/u(t) = -omega^2     =>  u(t) = cos(omega*t +
phi)\n");
printf("Axial piece:   H4''(z)/H4(z) = -kz^2      =>  H4(z) = Az*cos(kz*z)
+ Bz*sin(kz*z)\n\n");


printf("============================================================\n");
printf("SECTION 3 -- Radial equation (variable coefficient)\n");
printf("============================================================\n");

radial_ODE := diff(H3(r), r, r) + (1/r)*diff(H3(r), r)
              + (omega^2 * r / (R_sym * c_sym^2) - kz^2) * H3(r) = 0;
print(radial_ODE);

# Reduction H3 = g(r)/sqrt(r) removes the first-derivative term
printf("\nSubstitute H3(r) = g(r) / sqrt(r). The resulting equation for g
is:\n");
g_ODE := diff(g(r),r,r) + (omega^2*r/(R_sym*c_sym^2) - kz^2 + 1/(4*r^2)) *
g(r) = 0;
print(g_ODE);

# The 1/(4 r^2) term is subdominant for r >> sqrt(R c^2 / omega^2);
# dropping it yields the exact Airy reduction
printf("\nDropping the subdominant 1/(4 r^2) term:\n");
```

```
g_ODE_red := diff(g(r),r,r) + (omega^2*r/(R_sym*c_sym^2) - kz^2) * g(r) =
0;
print(g_ODE_red);


printf("================================================================\n");
printf("SECTION 4 -- Airy reduction: change of variable y(r)\n");
printf("================================================================\n");

# y = kappa * (r - kz^2 R c^2 / omega^2),  kappa = (omega^2/(R c^2))^(1/3)
# Then g_yy + y g = 0  =>  g(y) = C1 Ai(-y) + C2 Bi(-y)

kappa_expr := (omega^2 / (R_sym * c_sym^2))^(1/3);
y_of_r     := kappa_expr * (r - kz^2 * R_sym * c_sym^2 / omega^2);

printf("kappa = (omega^2/(R c^2))^(1/3)\n");
print(kappa_def = kappa_expr);
printf("y(r) = kappa * (r - kz^2 R c^2 / omega^2)\n");
print(y_def = y_of_r);

printf("\nIn variable y the equation reduces to Airy:\n");
printf("   g_yy(y) + y * g(y) = 0    =>   g_yy = (-y) * g\n");
printf("General solution:\n");
printf("   g(y) = C1 * AiryAi(-y) + C2 * AiryBi(-y)\n");
printf("Regularity at large r (where -y -> -inf) forces C2 = 0.\n\n");

# H3(r) = C1 / sqrt(r) * Ai(-y(r))
H3_form := H3(r) = C1 / sqrt(r) * AiryAi(-y_of_r);
print(H3_form);


printf("================================================================\n");
printf("SECTION 5 -- Boundary condition f(R,0,t)=0 -> spectrum\n");
printf("================================================================\n");

printf("Require H3(R) = 0  =>  AiryAi(-y(R)) = 0.\n");
printf("With kz = 0:  y(R) = (omega^2/(R c^2))^(1/3) * R = (omega R/c)^
(2/3)\n");
printf("Setting y(R) = |a_n|, the n-th positive zero of -arg of Ai:\n");
printf("   omega_n = (c/R) * |a_n|^(3/2)\n\n");

# Numerical Airy zeros (Ai(a_n) = 0, with a_n < 0)
# |a_n| values from Abramowitz/Stegun / DLMF
a_abs := [2.33810741, 4.08794944, 5.52055983,
          6.78670809, 7.94413359, 9.02265085,
          10.04017434, 11.00852430, 11.93601556, 12.82877675]:

printf("    n   |a_n|          |a_n|^(3/2)      omega_n [rad/s]     T_n [s]
\n");
```

```
printf("   ---  ----------   ------------   ------------------   ------
----------\n");
omega_list := []:
period_list := []:
for n from 1 to 10 do
   an := a_abs[n]:
   om := (c_val/R_val) * an^(3/2):
   Tn := 2*Pi / om:
   omega_list  := [op(omega_list),  om]:
   period_list := [op(period_list), Tn]:
   printf("    %2d   %.8f   %.8f   %.6e   %.6e\n", n, an, an^(3/2), om,
evalf(Tn));
end do:
printf("\n");


printf("============================================================\n");
printf("SECTION 6 -- Numerical verification of Airy eigenproblem\n");
printf("============================================================\n");

# For each n, with k_z = 0 we should have Ai(-y(R)) = 0 at omega =
omega_n.
# Verify by plugging back:
printf("Sanity check Ai(-y_n(R)) for n=1..5 (should be ~0):\n");
for n from 1 to 5 do
   om := omega_list[n]:
   kap := (om^2 / (R_val * c_val^2))^(1/3):
   yR := kap * R_val:
   val := evalf(AiryAi(-yR)):
   printf("   n=%d   y_n(R) = %.6e    Ai(-y_n(R)) = %.3e\n", n, yR, val);
end do:
printf("\n");


printf("============================================================\n");
printf("SECTION 7 -- Plots of radial eigenfunctions H3_n(r)\n");
printf("============================================================\n");

# Define the unnormalised radial eigenfunctions
H3n := proc(rr, n)
   local an, om, kap, yy;
   an := a_abs[n];
   om := omega_list[n];
   kap := (om^2 / (R_val * c_val^2))^(1/3);
   yy := kap * rr;        # k_z = 0
   evalf( AiryAi(-yy) / sqrt(rr) );
end proc:

# Plot the first three eigenmodes vs r / R
```

```
plot_H3_modes := plot(
   [seq( 'H3n(r * R_val, n) * sqrt(R_val)', n = 1..3 )],
   r = 0.001 .. 1.05,
   color = [red, blue, "DarkGreen"],
   thickness = 2,
   legend = ["n=1", "n=2", "n=3"],
   title = "Radial eigenfunctions H_3,n(r) * sqrt(R) (k_z=0)",
   labels = ["r / R", "H_3 sqrt(R)"]
):
print(plot_H3_modes);


# Plot of g_n(y) = Ai(-y) on the same y-axis
plot_Airy := plot(
   AiryAi(-y), y = 0 .. 15,
   color = "Purple", thickness = 2,
   title = "Airy function AiryAi(-y) : zeros at y = |a_n|",
   labels = ["y", "Ai(-y)"]
):
print(plot_Airy);


printf("============================================================\n");
printf("SECTION 8 -- Maxwell-type system for P(r,z,t)\n");
printf("============================================================\n");

printf("System:  P_tt - c^2 P_rr = 0,  cylindrical symmetry, P(R,0,0)=0.
\n");
printf("Ansatz (field, then P = |psi|^2):\n");
printf("   psi(r,z,t) = N(z) * exp(-(r-c t)^2 / (2 sigma_r^2)) * cos(k_r
(r - c t))\n");
printf("   P(r,z,t)   = N(z)^2 * exp(-(r-c t)^2 / sigma_r^2)   * cos^2(k_r
(r - c t))\n\n");

# Symbolic verification: any function of xi = r - c t satisfies P_tt = c^2
P_rr
P_ansatz := exp(-(r - c_sym*t)^2/sigma_r^2) * cos(k_r*(r - c_sym*t))^2;
res := simplify(diff(P_ansatz, t, t) - c_sym^2 * diff(P_ansatz, r, r));
printf("Symbolic check of  P_tt - c^2 P_rr  for P(r,t) above:\n");
print(res);
printf("\n=> identically 0; the d'Alembert equation is satisfied.\n\n");


printf("============================================================\n");
printf("SECTION 9 -- Periodic zeros of P at fixed t (lattice r_b)\n");
printf("============================================================\n");

printf("Zeros of P(., t0) require cos(k_r (r - c t0)) = 0\n");
printf("   =>  k_r (r - c t0) = (m + 1/2) pi\n");
```

```
printf("   =>  consecutive zeros separated by  r_b = pi / k_r\n\n");

printf("Boundary condition P(R,0,0) = 0 with phi_z = 0 gives:\n");
printf("   cos(k_r * R) = 0  =>  k_r * R = (n + 1/2) pi\n");
printf("   =>  r_b = R / (n + 1/2),     n = 0, 1, 2, ...\n\n");

printf("    n    r_b/R           r_b [m]\n");
printf("   ---   -----------     ------------------\n");
for n from 0 to 6 do
   rb_over_R := evalf(1/(n + 1/2)):
   rb_m      := rb_over_R * R_val:
   printf("    %d    %.10f    %.6e\n", n, rb_over_R, rb_m);
end do:

printf("\n");
printf("Natural match to lowest Airy mode (k_r = omega_1/c = |a_1|^(3/2)
/R):\n");
k_r_nat  := evalf(a_abs[1]^(3/2) / R_val):
rb_nat   := evalf(Pi / k_r_nat):
printf("   k_r_nat = %.6e 1/m\n", k_r_nat);
printf("   r_b_nat = %.6e m   =   %.6f * R\n", rb_nat, rb_nat/R_val);
printf("   (lies between n=0 (r_b=2R) and n=1 (r_b=2R/3) BC values)\n\n");


printf("============================================================\n");
printf("SECTION 10 -- Plots of P(r,z=0,t) and verification\n");
printf("============================================================\n");

# Choose the n=1 BC-compatible value: r_b = 2R/3, so k_r = 3 pi / (2 R)
n_choice  := 1:
r_b_plot  := R_val / (n_choice + 1/2):    # = 2R/3 for n=1
k_r_plot  := Pi / r_b_plot:
sigma_r   := 0.4 * R_val:                  # envelope width

P_num := (rr, tt) -> evalf(
   exp(-(rr - c_val*tt)^2 / sigma_r^2)
   * cos(k_r_plot * (rr - c_val*tt))^2
):

# (a) P at fixed t = 0 vs r (showing periodic zeros at multiples of r_b/2
offset)
plot_P_t0 := plot(
   'P_num(r * R_val, 0)', r = 0 .. 2.0,
   thickness = 2, color = "DarkRed",
   title = sprintf("P(r,0,0) with k_r = 3π/(2R), σ_r = 0.4 R (n=%d BC
mode)", n_choice),
   labels = ["r / R", "P(r,0,0)"]
):
print(plot_P_t0);
```

```
# (b) Verify P(R,0,0) = 0 numerically
P_at_R := P_num(R_val, 0):
printf("Verification:  P(R, 0, 0) = %.3e   (should be ~ 0)\n", P_at_R);

# (c) Verify zero spacing equals r_b
# Find approximate zero locations near r/R = 0.5..1.5
printf("\nApproximate zero locations of P(.,0,0) near r ~ R:\n");
for m from -3 to 3 do
   rz := (m + 1/2) * r_b_plot:    # zeros at (m+1/2) * r_b
   if rz > 0 then
      printf("   m=%2d   r/R = %.6f    r = %.4e m\n", m, rz/R_val, rz);
   end if;
end do:

# (d) Travelling wave: P(r,0,t) for two times
plot_P_travel := plot(
   ['P_num(r * R_val, 0)',
    'P_num(r * R_val, 0.3 * T_R)'],
   r = 0 .. 2.0,
   color = ["DarkRed", "DarkBlue"],
   thickness = 2,
   legend = ["t = 0", "t = 0.3 R/c"],
   title = "P(r,0,t): Gaussian-modulated wave packet, travels at speed c",
   labels = ["r / R", "P"]
):
print(plot_P_travel);

# (e) Density plot of P(r, t) in the (r,t) plane
plot_P_density := densityplot(
   P_num(r * R_val, t * T_R),
   r = 0 .. 2.0, t = -0.5 .. 1.5,
   grid = [120, 120],
   colorstyle = HUE,
   axes = boxed,
   title = "Density of P(r,0,t) — characteristic lines r = c t + const",
   labels = ["r / R", "t / (R/c)"]
):
print(plot_P_density);


printf("\n============================================================\n")
;
printf("SECTION 11 -- Reconstruction H(r,theta,z,t) (axisymmetric)\n");
printf("============================================================\n");

printf("The axisymmetric (no-theta) field assembled from (17)-(18):\n\n");
printf("   H(r,theta,z,t) = sum_{n,kz}  C_{n,kz} * AiryAi(-y_{n,kz}(r))
/sqrt(r)\n");
printf("                                        * cos(kz z + phi_{kz})\n");
```

```
printf("                                          * cos(omega_n t + phi_n)
\n\n");
printf("with y_{n,kz}(r) = (omega_n^2/(R c^2))^(1/3) * (r - kz^2 R
c^2/omega_n^2).\n");
printf("The field has no theta dependence, by construction.\n\n");

# Visualise one composite mode: n=1, kz=0, integrated over z trivially
H_one := proc(rr, zz, tt, n, kz_val)
   local an, om, kap, yy;
   an := a_abs[n];
   om := omega_list[n];
   kap := (om^2 / (R_val * c_val^2))^(1/3);
   yy := kap * (rr - kz_val^2 * R_val * c_val^2 / om^2);
   evalf( AiryAi(-yy) / sqrt(rr) * cos(kz_val * zz) * cos(om * tt) );
end proc:

plot_H_mode1 := plot3d(
   'H_one(r * R_val, 0, t * T_R, 1, 0) * sqrt(R_val)',
   r = 0.05 .. 1.0, t = 0 .. 2.0,
   grid = [60, 60],
   axes = boxed,
   shading = ZHUE,
   title = "Lowest cylindrical eigenmode H_{n=1, k_z=0}(r,0,t) * sqrt(R)",
   labels = ["r / R", "t / (R/c)", "H"]
):
print(plot_H_mode1);


printf("============================================================\n");
printf("SECTION 12 -- Summary table\n");
printf("============================================================\n");

printf("Quantity                          Value / formula\n");
printf("-----------------------------     -------------------------------
-\n");
printf("R                                 %.6e m\n", R_val);
printf("c                                 %.6e m/s\n", c_val);
printf("R/c                               %.6e s\n", T_R);
printf("Lowest Airy zero |a_1|            %.8f\n", a_abs[1]);
printf("|a_1|^(3/2)                       %.6f\n", a_abs[1]^(3/2));
printf("omega_1 (k_z=0)                   %.6e rad/s\n", omega_list[1]);
printf("Period T_1                        %.6e s\n", evalf(period_list[1])
);
printf("BC r_b spectrum                   r_b = R / (n + 1/2),  n=0,1,2,..
.\n");
printf("r_b(n=0)                          %.4e m   (= 2 R)\n", 2*R_val);
printf("r_b(n=1)                          %.4e m   (= 2R/3)\n", 2*R_val/3)
;
printf("r_b_natural (mode-1 match)        %.4e m   (= %.5f R)\n", rb_nat,
```

```
  rb_nat/R_val);

  printf("\n============================================================\n")
  ;
  printf("End of transcript.\n");
  printf("============================================================\n");
```

```
============================================================
SECTION 0 -- Physical constants and TauTimp anchors
============================================================
R        = 3.922800e+22 m
c        = 2.997925e+08 m/s
R/c      = 1.308505e+14 s
R^2/c^2  = 1.712186e+28 s^2

============================================================
SECTION 1 -- Cylindrical Laplacian (axisymmetric, no theta)
============================================================
```

$$master_PDE := \frac{R_sym\left(\frac{\partial^2}{\partial r^2}F(r,z,t) + \frac{\frac{\partial}{\partial r}F(r,z,t)}{r} + \frac{\partial^2}{\partial z^2}F(r,z,t)\right)}{r} = \frac{\frac{\partial^2}{\partial t^2}F(r,z,t)}{c_sym^2}$$

$$\frac{R_sym\left(\frac{\partial^2}{\partial r^2}F(r,z,t) + \frac{\frac{\partial}{\partial r}F(r,z,t)}{r} + \frac{\partial^2}{\partial z^2}F(r,z,t)\right)}{r} = \frac{\frac{\partial^2}{\partial t^2}F(r,z,t)}{c_sym^2}$$

```
============================================================
SECTION 2 -- Separation f = H3(r) H4(z) u(t)
============================================================
```

$$PDE_sub := \frac{u(t)\,R_sym\left(H3(r)\left(\frac{d^2}{dz^2}H4(z)\right)r + H4(z)\left(r\left(\frac{d^2}{dr^2}H3(r)\right) + \frac{d}{dr}H3(r)\right)\right)}{r^2} - \frac{H3(r)\,H4(z)\left(\frac{d^2}{dt^2}u(t)\right)}{c_sym^2}$$

```
After dividing by H3*H4*u, the separated form is:

  (R/r)[ H3''/H3 + (1/r) H3'/H3 + H4''/H4 ] = (1/c^2) u''/u

Time piece:    u''(t)/u(t) = -omega^2      =>  u(t) = cos(omega*t + phi)
Axial piece:   H4''(z)/H4(z) = -kz^2       =>  H4(z) = Az*cos(kz*z) + Bz*sin
(kz*z)
```

```
============================================================
SECTION 3 -- Radial equation (variable coefficient)
============================================================
```

$$radial_ODE := \frac{d^2}{dr^2} H3(r) + \frac{\frac{d}{dr} H3(r)}{r} + \left(\frac{\omega^2 r}{R_sym\, c_sym^2} - kz^2 \right) H3(r) = 0$$

$$\frac{d^2}{dr^2} H3(r) + \frac{\frac{d}{dr} H3(r)}{r} + \left(\frac{\omega^2 r}{R_sym\, c_sym^2} - kz^2 \right) H3(r) = 0$$

Substitute H3(r) = g(r) / sqrt(r). The resulting equation for g is:

$$g_ODE := \frac{d^2}{dr^2} g(r) + \left(\frac{\omega^2 r}{R_sym\, c_sym^2} - kz^2 + \frac{1}{4 r^2} \right) g(r) = 0$$

$$\frac{d^2}{dr^2} g(r) + \left(\frac{\omega^2 r}{R_sym\, c_sym^2} - kz^2 + \frac{1}{4 r^2} \right) g(r) = 0$$

Dropping the subdominant 1/(4 r^2) term:

$$g_ODE_red := \frac{d^2}{dr^2} g(r) + \left(\frac{\omega^2 r}{R_sym\, c_sym^2} - kz^2 \right) g(r) = 0$$

$$\frac{d^2}{dr^2} g(r) + \left(\frac{\omega^2 r}{R_sym\, c_sym^2} - kz^2 \right) g(r) = 0$$

```
============================================================
SECTION 4 -- Airy reduction: change of variable y(r)
============================================================
```

$$kappa_expr := \left(\frac{\omega^2}{R_sym\, c_sym^2} \right)^{1/3}$$

$$y_of_r := \left(\frac{\omega^2}{R_sym\, c_sym^2} \right)^{1/3} \left(r - \frac{kz^2\, R_sym\, c_sym^2}{\omega^2} \right)$$

kappa = (omega^2/(R c^2))^(1/3)

$$kappa_def = \left(\frac{\omega^2}{R_sym\, c_sym^2} \right)^{1/3}$$

y(r) = kappa * (r - kz^2 R c^2 / omega^2)

$$y_def = \left(\frac{\omega^2}{R_sym\, c_sym^2} \right)^{1/3} \left(r - \frac{kz^2\, R_sym\, c_sym^2}{\omega^2} \right)$$

```
In variable y the equation reduces to Airy:
   g_yy(y) + y * g(y) = 0    =>   g_yy = (-y) * g
General solution:
   g(y) = C1 * AiryAi(-y) + C2 * AiryBi(-y)
Regularity at large r (where -y -> -inf) forces C2 = 0.
```

$$H3_form := H3(r) = \frac{C1\ \mathrm{AiryAi}\left(-\left(\frac{\omega^2}{R_sym\ c_sym^2}\right)^{1/3}\left(r-\frac{kz^2\ R_sym\ c_sym^2}{\omega^2}\right)\right)}{\sqrt{r}}$$

$$H3(r) = \frac{C1\ \mathrm{AiryAi}\left(-\left(\frac{\omega^2}{R_sym\ c_sym^2}\right)^{1/3}\left(r-\frac{kz^2\ R_sym\ c_sym^2}{\omega^2}\right)\right)}{\sqrt{r}}$$

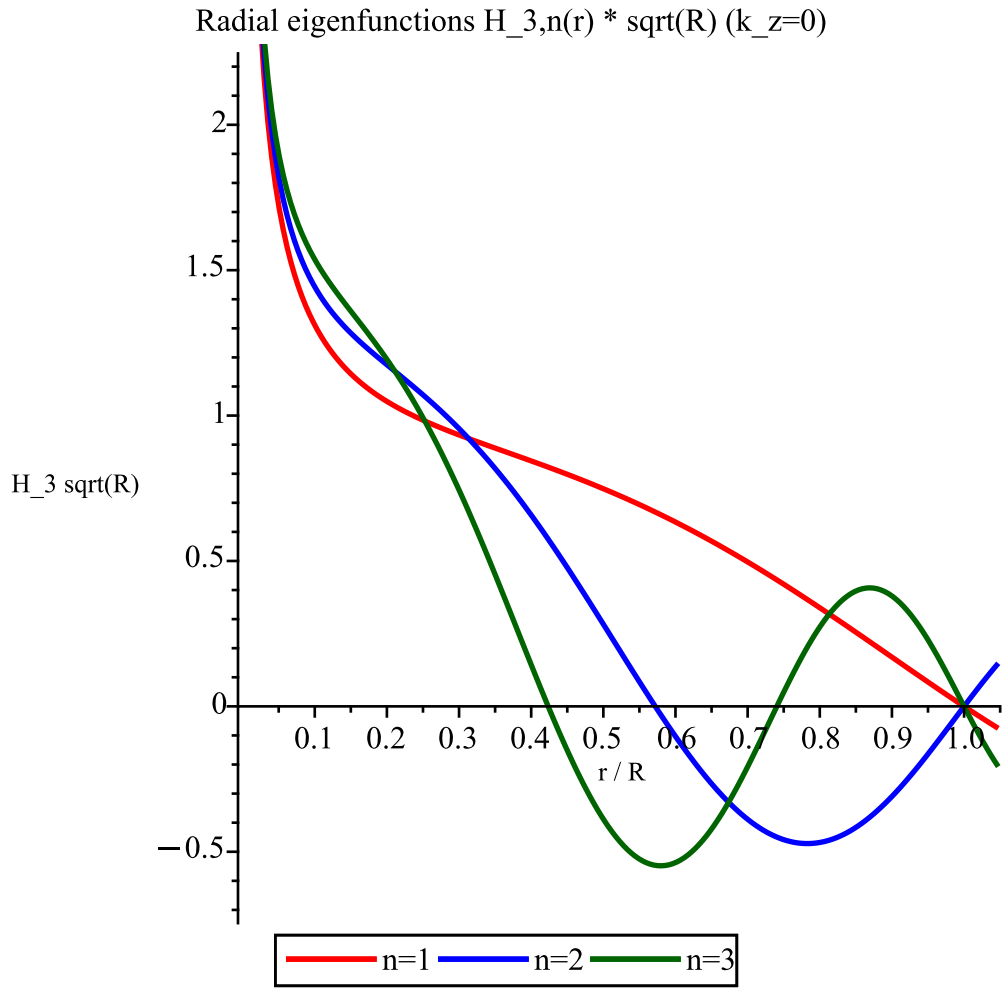
```
============================================================
SECTION 5 -- Boundary condition f(R,0,t)=0 -> spectrum
============================================================
Require H3(R) = 0  =>  AiryAi(-y(R)) = 0.
With kz = 0:  y(R) = (omega^2/(R c^2))^(1/3) * R = (omega R/c)^(2/3)
Setting y(R) = |a_n|, the n-th positive zero of -arg of Ai:
   omega_n = (c/R) * |a_n|^(3/2)

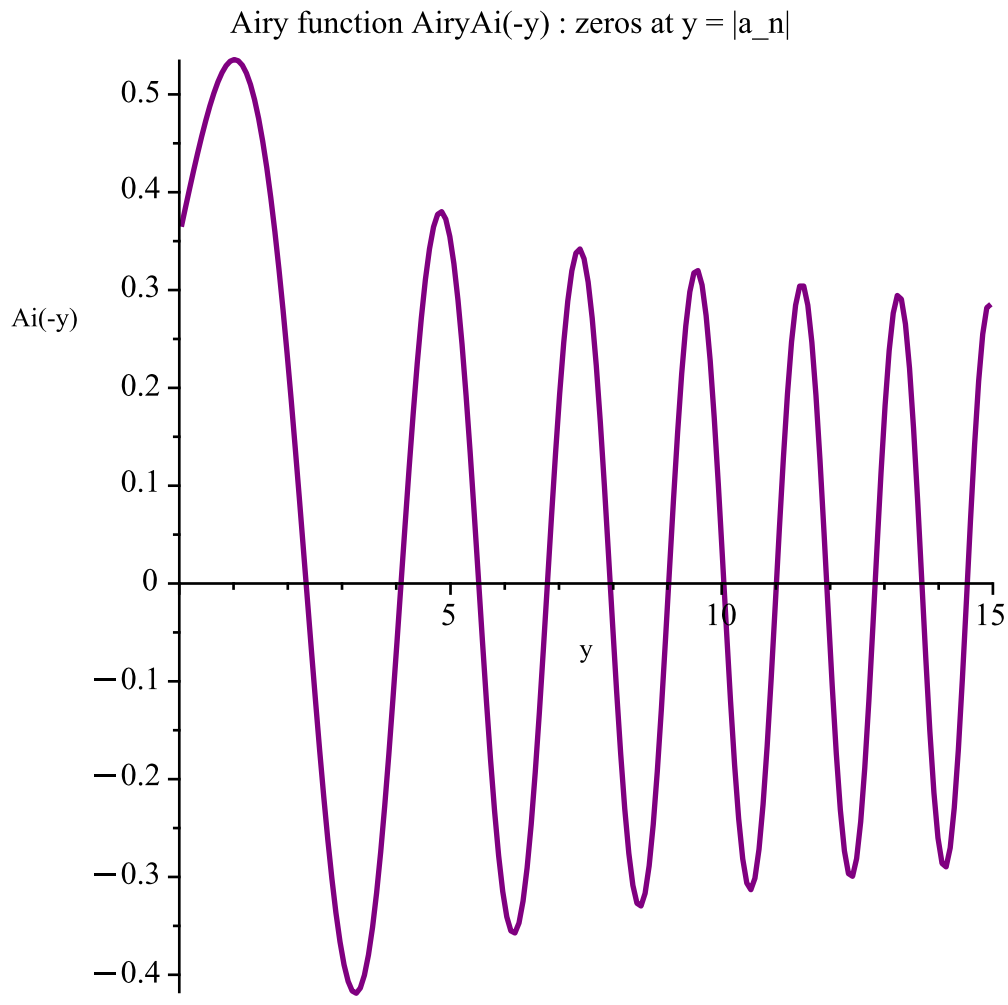
    n   |a_n|          |a_n|^(3/2)      omega_n [rad/s]     T_n [s]
   ---  -----------    -------------   -------------------   ---------------
     1   2.33810741    3.57516992   2.732255e-14   2.299634e+14
     2   4.08794944    8.26529338   6.316592e-14   9.947113e+13
     3   5.52055983   12.97103651    9.912866e-14    6.338415e+13
     4   6.78670809   17.68026437   1.351180e-13   4.650146e+13
     5   7.94413359   22.39081017   1.711175e-13   3.671855e+13
     6   9.02265085   27.10199292   2.071218e-13   3.033571e+13
     7   10.04017434    31.81353149    2.431288e-13    2.584303e+13
     8   11.00852430    36.52528876    2.791375e-13    2.250928e+13
     9   11.93601556    41.23719006    3.151473e-13    1.993730e+13
    10   12.82877675    45.94919118    3.511579e-13    1.789277e+13

============================================================
SECTION 6 -- Numerical verification of Airy eigenproblem
============================================================
Sanity check Ai(-y_n(R)) for n=1..5 (should be ~0):
   n=1   y_n(R) = 2.338107e+00    Ai(-y_n(R)) = -1.080e-09
   n=2   y_n(R) = 4.087949e+00    Ai(-y_n(R)) = -2.515e-09
   n=3   y_n(R) = 5.520560e+00    Ai(-y_n(R)) = -4.243e-09
   n=4   y_n(R) = 6.786708e+00    Ai(-y_n(R)) = 8.455e-10
   n=5   y_n(R) = 7.944134e+00    Ai(-y_n(R)) = -1.780e-09

============================================================
SECTION 7 -- Plots of radial eigenfunctions H3_n(r)
```

==========


Radial eigenfunctions H_3,n(r) * sqrt(R) (k_z=0)
H_3 sqrt(R)
2
1.5
1
0.5
0
−0.5
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
r / R
n=1
n=2
n=3


Airy function AiryAi(-y) : zeros at y = |a_n|
Ai(-y)
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
5
10
15
y

```
============================================================
SECTION 8 -- Maxwell-type system for P(r,z,t)
============================================================
System:  P_tt - c^2 P_rr = 0,  cylindrical symmetry, P(R,0,0)=0.
Ansatz (field, then P = |psi|^2):
   psi(r,z,t) = N(z) * exp(-(r-c t)^2 / (2 sigma_r^2)) * cos(k_r (r - c t))
   P(r,z,t)   = N(z)^2 * exp(-(r-c t)^2 / sigma_r^2)   * cos^2(k_r (r - c t))
```

$$P_ansatz := e^{-\frac{(-c_sym\,t+r)^2}{sigma_r^2}} \cos\left(k_r\left(-c_sym\,t+r\right)\right)^2$$

$$res := 0$$

```
Symbolic check of  P_tt - c^2 P_rr  for P(r,t) above:
```

$$0$$

```
=> identically 0; the d'Alembert equation is satisfied.

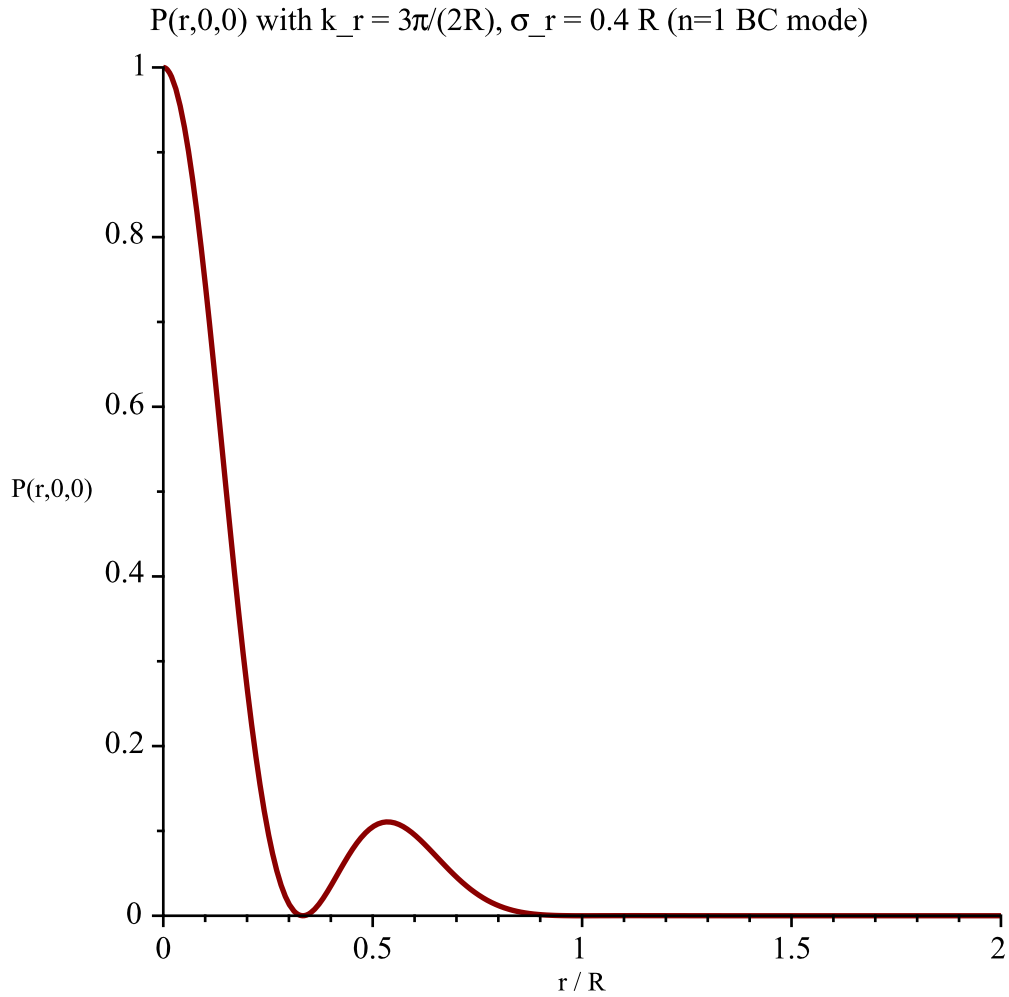
============================================================
SECTION 9 -- Periodic zeros of P at fixed t (lattice r_b)
============================================================
Zeros of P(., t0) require cos(k_r (r - c t0)) = 0
   =>  k_r (r - c t0) = (m + 1/2) pi
   =>  consecutive zeros separated by  r_b = pi / k_r

Boundary condition P(R,0,0) = 0 with phi_z = 0 gives:
   cos(k_r * R) = 0  =>  k_r * R = (n + 1/2) pi
   =>  r_b = R / (n + 1/2),     n = 0, 1, 2, ...

    n    r_b/R           r_b [m]
   ---   -----------     ------------------
    0    2.0000000000    7.845600e+22
    1    0.6666666667    2.615200e+22
    2    0.4000000000    1.569120e+22
    3    0.2857142857    1.120800e+22
    4    0.2222222222    8.717333e+21
    5    0.1818181818    7.132364e+21
    6    0.1538461538    6.035077e+21

Natural match to lowest Airy mode (k_r = omega_1/c = |a_1|^(3/2)/R):
   k_r_nat = 9.113822e-23 1/m
   r_b_nat = 3.447064e+22 m   =   0.878725 * R
   (lies between n=0 (r_b=2R) and n=1 (r_b=2R/3) BC values)

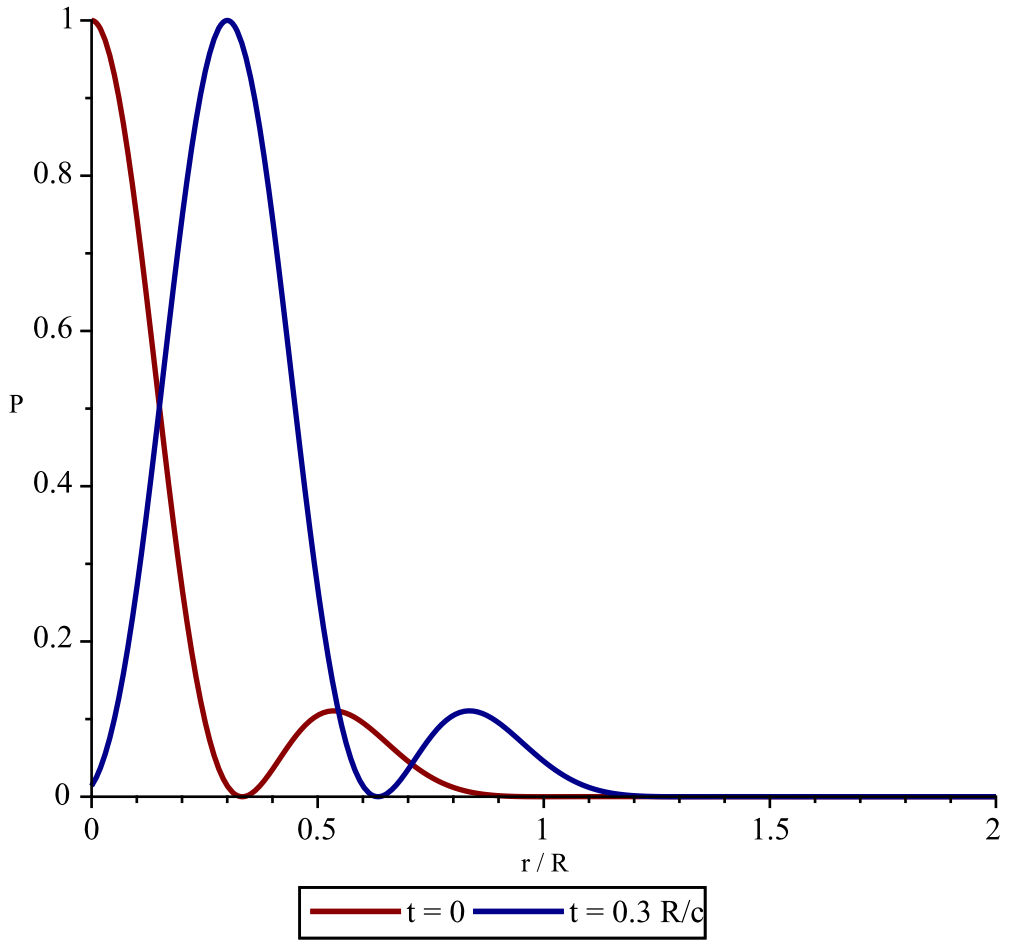
============================================================
SECTION 10 -- Plots of P(r,z=0,t) and verification
============================================================
```



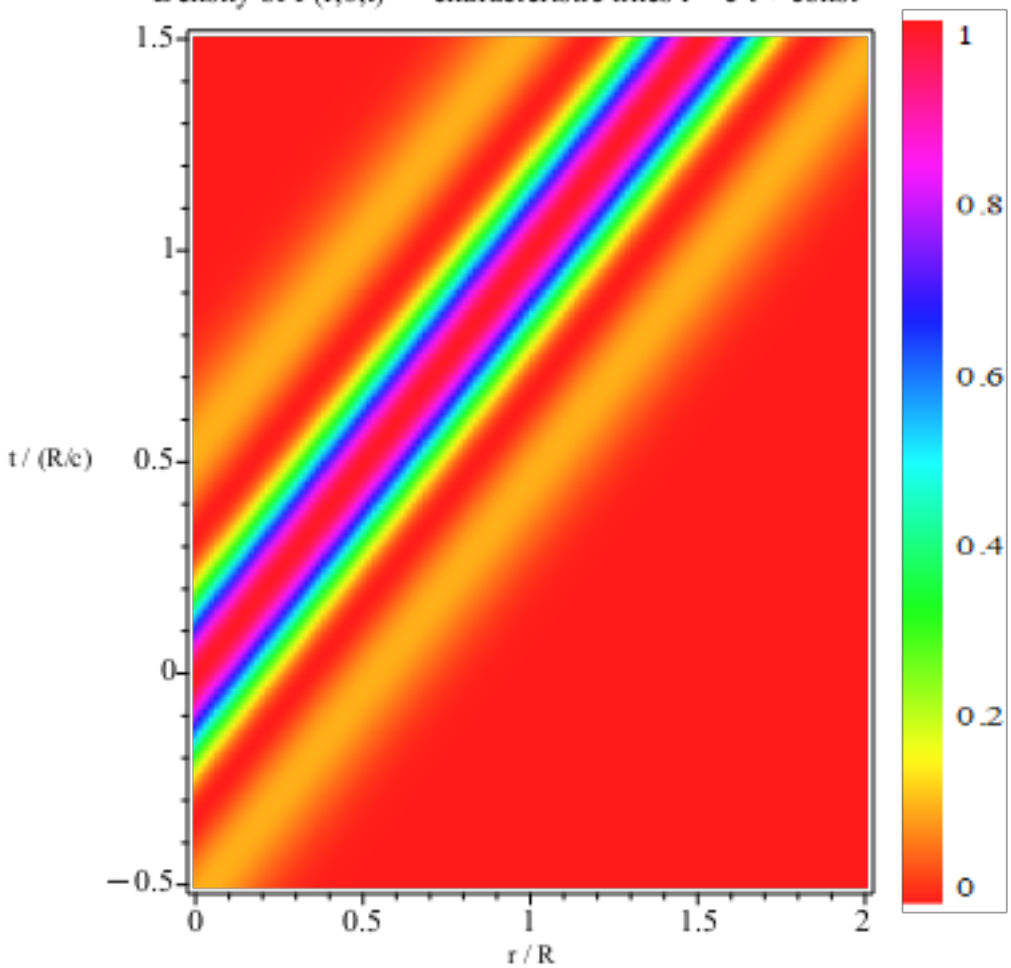

```
Verification:  P(R, 0, 0) = 1.320e-20   (should be ~ 0)

Approximate zero locations of P(.,0,0) near r ~ R:
   m= 0   r/R = 0.333333    r = 1.3076e+22 m
   m= 1   r/R = 1.000000    r = 3.9228e+22 m
   m= 2   r/R = 1.666667    r = 6.5380e+22 m
   m= 3   r/R = 2.333333    r = 9.1532e+22 m
```

P(r,0,t): Gaussian-modulated wave packet, travels at speed c



Density of P(r,0,t) — characteristic lines r = c t + const



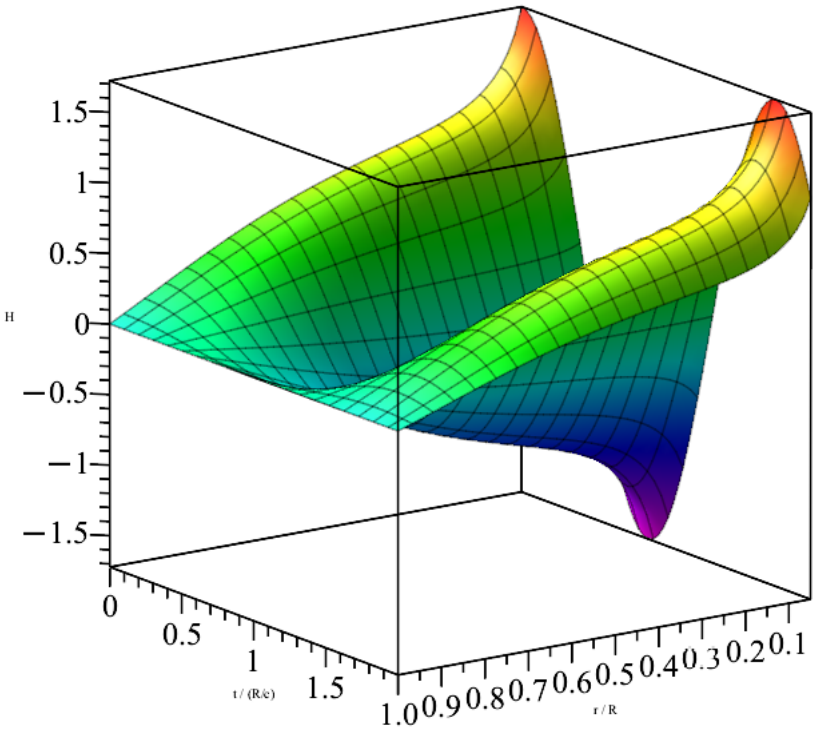
==========

```
SECTION 11 -- Reconstruction H(r,theta,z,t) (axisymmetric)
============================================================
The axisymmetric (no-theta) field assembled from (17)-(18):

   H(r,theta,z,t) = sum_{n,kz}  C_{n,kz} * AiryAi(-y_{n,kz}(r))/sqrt(r)
                                         * cos(kz z + phi_{kz})
                                         * cos(omega_n t + phi_n)

with y_{n,kz}(r) = (omega_n^2/(R c^2))^(1/3) * (r - kz^2 R c^2/omega_n^2).
The field has no theta dependence, by construction.
```

Lowest cylindrical eigenmode H_{n=1, k_z=0}(r,0,t) * sqrt(R)




```
============================================================
SECTION 12 -- Summary table
============================================================
```

Quantity	Value / formula
R	3.922800e+22 m
c	2.997925e+08 m/s
R/c	1.308505e+14 s
Lowest Airy zero |a_1|	2.33810741
|a_1|^(3/2)	3.575170
omega_1 (k_z=0)	2.732255e-14 rad/s
Period T_1	2.299634e+14 s
BC r_b spectrum	r_b = R / (n + 1/2), n=0,1,2,...

```
r_b(n=0)                              7.8456e+22 m    (= 2 R)
r_b(n=1)                              2.6152e+22 m    (= 2R/3)
r_b_natural (mode-1 match)            3.4471e+22 m    (= 0.87873 R)

============================================================
End of transcript.
============================================================
```